

CSC-3TC33 : DEVOIR MAISON n° 2

2.1. Enveloppe convexe et parcours de Graham

Dans cet exercice, on se place dans le plan euclidien orienté usuel $\mathbb{R}^2$; les éléments du plan sont représentés par des couples (x, y) formés d'une abscisse x et d'une ordonnée y .

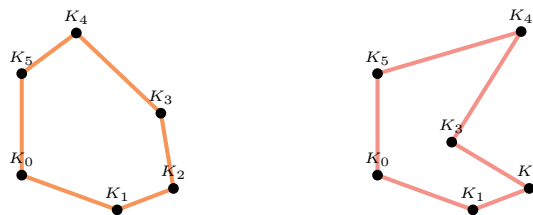
1. Rappeler la formule analytique du produit vectoriel $\vec{u} \wedge \vec{v}$ entre deux vecteurs du plan $\vec{u} = (u_x, u_y)$ et $\vec{v} = (v_x, v_y)$.

Soient A, B et C trois points du plan. On dit que le triplet (A, B, C) forme un *virage à gauche* si l'angle algébrique $\widehat{(\overrightarrow{AB}, \overrightarrow{BC})}$ est positif ; on dit que le triplet (A, B, C) forme un *virage à droite* si l'angle algébrique $\widehat{(\overrightarrow{AB}, \overrightarrow{BC})}$ est négatif.



2. Montrer que l'on peut détecter qu'un virage à gauche par un test de signe sur un produit vectoriel bien choisi.

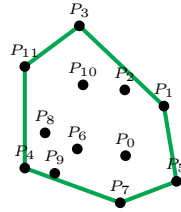
On considère une suite de points distincts $(K_i)_{0 \leq i < h} \subseteq \mathbb{R}^2$. Si nécessaire, on appelle K_i le point K_{i_0} où $i_0 \in \llbracket 0, h \rrbracket$ est le reste de i modulo h .



3. Caractériser le fait que les sommets $(K_i)_{0 \leq i < h} \in \mathbb{R}^2$ délimitent un polygone convexe dans le sens positif (le sens trigonométrique) par des considérations sur les triplets $(K_i, K_{i+1}, K_{i+2})_{0 \leq i < h} \in \mathbb{R}^2$

Nous appelons *enveloppe convexe* d'une famille de points $(P_j)_{0 \leq j < n} \subseteq \mathbb{R}^2$ le plus petit convexe contenant la famille de points, soit encore l'ensemble des combinaisons convexe des points $(P_j)_{0 \leq j < n} \subseteq \mathbb{R}^2$:

$$C = \left\{ \sum_{i=1}^n \lambda_j M_j \in \mathbb{R}^2; (\lambda_j)_{0 \leq j < n} \in [0, 1]^n \text{ t.q. } \sum_{j=1}^n \lambda_j = 1 \right\}.$$



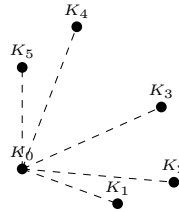
Nous rappelons que, comme $(P_j)_{0 \leq j < n} \subseteq \mathbb{R}^2$ est une famille finie, son enveloppe convexe C est en vérité un polygone $(K_i)_{0 \leq i < h}$ où chacun des points K_i est tiré de la famille $(P_j)_{0 \leq j < n}$. Par exemple, dans le croquis ci-dessus, il s'agit de l'hexagone $(P_4 P_7 P_5 P_1 P_3 P_{11})$.

Soit $\hat{P}$ le plus petit sommet de la famille $(P_j)_{0 \leq j < n}$ au sens de l'ordre lexicographique de $\mathbb{R}^2$.

4. Montrer que $\hat{P}$ appartient à la frontière de C

Dans ce qui suit, nous supposons que $K_0 = \hat{P}$.

5. Caractériser, pour tous indices $0 < i < j < h$, la situation du triplet (K_i, K_0, K_j) .

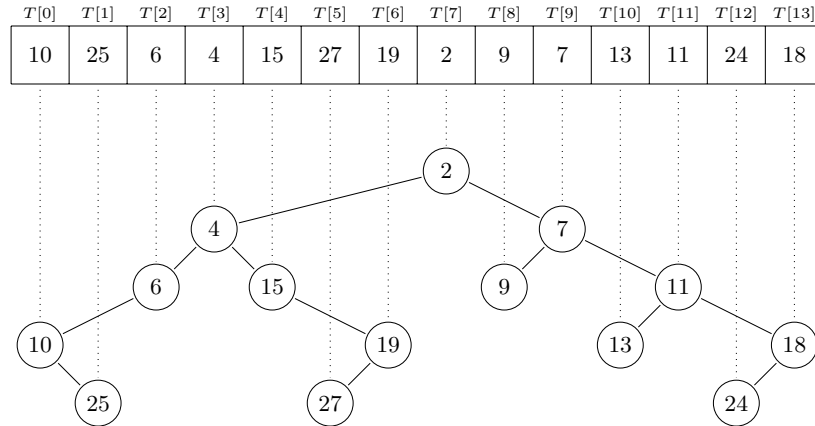


6. En déduire une manière de trier les points $(P_j)_{1 \leq j < n}$ tels que la suite $(K_i)_{1 \leq i < h} \in \mathbb{R}^2$ en soit une suite extraite.
7. Proposer un algorithme de calcul de l'enveloppe convexe C qui examine chacun des points $(P_j)_{1 \leq j < n}$ triés dans l'ordre de la question 6. Cet algorithme manipule une collection de points Π (par des insertions ou suppressions) qui obéit à l'invariant : Π contient l'enveloppe convexe des points déjà examinés. On identifiera quelles opérations sont requises sur la collection Π et on précisera le type abstrait adapté.
8. Implémenter l'algorithme en Python pour vérifier que vos idées fonctionnent.

2.2. Arbres cartésiens

On dit qu'un arbre binaire étiqueté jouit de la *propriété de tas* si l'étiquette de tout sommet interne est inférieure à celle de ses fils⁽¹⁾.

Soit T un tableau contenant des éléments $(a_i)_{0 \leq i < n}$ tous distincts et comparables entre eux. On appelle *arbre cartésien* tout arbre binaire jouissant de la propriété de tas tel que la suite de ses étiquettes parcourues dans l'ordre symétrique est égale à la suite $(a_i)_{0 \leq i < n}$.



On représente un arbre cartésien par un tableau $\hat{T}$ de dimension $4 \times n$: la première ligne de $\hat{T}$ contient les valeurs de $(a_i)_{0 \leq i < n}$, la seconde ligne contient les indices du père, la troisième et quatrième ligne contiennent les indices du fils gauche et du fils droit. La valeur -1 représente une valeur sentinelle indiquant l'absence d'un sommet.

$\hat{T}[0]$	$\hat{T}[1]$	$\hat{T}[2]$	$\hat{T}[3]$	$\hat{T}[4]$	$\hat{T}[5]$	$\hat{T}[6]$	$\hat{T}[7]$	$\hat{T}[8]$	$\hat{T}[9]$	$\hat{T}[10]$	$\hat{T}[11]$	$\hat{T}[12]$	$\hat{T}[13]$
10	25	6	4	15	27	19	2	9	7	13	12	24	18
2	0	3	7	3	6	4	-1	9	7	11	9	13	11
-1	-1	0	2	-1	-1	5	3	-1	8	-1	10	-1	12
-1	-1	0	-1	6	-1	-1	9	-1	11	-1	13	-1	-1

1. Montrer que, s'il existe, l'arbre cartésien d'un tableau (de dimension $1 \times n$) est unique.
2. On suppose que, pour un certain entier $m \in \llbracket 0, n-1 \rrbracket$, un arbre cartésien A_m a été construit à partir des éléments $(a_i)_{0 \leq i < m}$. Décrire un algorithme qui construit un arbre cartésien A_{m+1} contenant les éléments $(a_i)_{0 \leq i < m+1}$. Donner la complexité de cet algorithme.

1. Il ne s'agit pas forcément d'un *tas* puisque l'arbre n'est pas forcément parfait.

On appelle $\mu(i, j)$ le minimum

$$\mu(i, j) = \min_{i \leq k \leq j} a_k.$$

3. Caractériser la position du sommet d'étiquette $\mu(i, j)$ dans l'arbre cartésien du tableau T .

2.3. Loi de De Morgan

Dans le cours, nous avons défini l'ensemble $\mathcal{F}$ des formules de logique par un système d'induction dont nous rappelons les règles :

$$\begin{array}{c} \frac{}{x_n \in \mathcal{F}} \text{ (Variable } n) \\ \frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{(\varphi \vee \psi) \in \mathcal{F}} \text{ (Ou)} \\ \frac{\varphi \in \mathcal{F}}{\neg \varphi \in \mathcal{F}} \text{ (Negation)} \\ \frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{(\varphi \wedge \psi) \in \mathcal{F}} \text{ (Et)} \end{array}$$

où n appartient à $\mathbb{N}$.

1. Montrer que le mot $w_0 = ((x_1 \wedge \neg x_5) \vee \neg(\neg x_3 \vee (x_4 \wedge x_2)))$ appartient à l'ensemble $\mathcal{F}$ en exhibant un arbre de dérivation.

Nous définissons une fonction $\tau : \mathcal{F} \rightarrow \mathcal{F}$ par les cas suivants :

$$\begin{array}{c} \frac{}{\tau(x_n) = \neg x_n} \text{ (Variable } n) \\ \frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{\tau((\varphi \vee \psi)) = (\tau(\varphi) \wedge \tau(\psi))} \text{ (Ou)} \\ \frac{\varphi \in \mathcal{F}}{\tau(\neg \varphi) = \neg \tau(\varphi)} \text{ (Negation)} \\ \frac{\varphi \in \mathcal{F} \quad \psi \in \mathcal{F}}{\tau((\varphi \wedge \psi)) = (\tau(\varphi) \vee \tau(\psi))} \text{ (Et)} \end{array}$$

2. Calculer $\tau(w_0)$ où la formule w_0 a été introduite à la question précédente.

Soit $\alpha : \mathbb{N} \rightarrow \mathbb{B}$ une valuation des variables booléennes. Pour tout $w \in \mathcal{F}$, nous notons, comme dans le cours, $\llbracket w \rrbracket_\alpha$ l'évaluation de w d'après α .

3. Démontrer que pour tout mot $w \in \mathcal{F}$, on a

$$\llbracket \tau(w) \rrbracket_\alpha = \llbracket \neg w \rrbracket_\alpha$$

2.4. L'algorithme de Quine

Dans cet exercice, la notation $\overline{\mathcal{F}}$ désigne l'ensemble formules de logique étendues, qui est défini inductivement par les quatre règles rappelée dans l'exercice précédent et deux nouvelles règles :

$$\frac{}{\top \in \overline{\mathcal{F}}} \text{ (Tautologie)} \quad \frac{}{\perp \in \overline{\mathcal{F}}} \text{ (Antilogie)}$$

Nous complétons la définition de la fonction d'évaluation par les cas suivants :

$$\frac{}{\llbracket \top \rrbracket_\alpha = V} \text{ (Tautologie)} \quad \frac{}{\llbracket \perp \rrbracket_\alpha = F} \text{ (Antilogie)}$$

Définition 2.1. — Nous disons qu'une formule ϕ est *satisfiable* s'il existe une valuation α telle que $\llbracket \phi \rrbracket_\alpha = V$.

1. En supposant qu'il apparaît exactement n variables booléennes dans la formule ϕ , combien d'appels à la fonction d'évaluation $\phi \mapsto \llbracket \phi \rrbracket_\alpha$ sont-ils nécessaires dans le pire des cas pour déterminer si une formule est satisfiable ?

2. Décrire une fonction $\sigma : \overline{\mathcal{F}} \rightarrow \mathcal{F} \cup \{\top, \perp\}$ telle que, pour toute formule étendue ϕ , on ait

$$\llbracket \phi \rrbracket_\alpha = \llbracket \sigma(\phi) \rrbracket_\alpha$$

Soit $\phi \in \overline{\mathcal{F}}$ une formule étendue. Nous notons $\phi[x_n \leftarrow \top]$ la formule de $\overline{\mathcal{F}}$ obtenue en substituant toute occurrence de la variable x_n par le symbole $\top$; nous notons $\phi[x_n \leftarrow \perp]$ la formule de $\overline{\mathcal{F}}$ obtenue en substituant toute occurrence de la variable x_n par le symbole $\perp$.

3. Montrer que la formule ϕ est satisfiable si et seulement si $\phi[x_n \leftarrow \top]$ est satisfiable ou $\phi[x_n \leftarrow \perp]$ est satisfiable.
4. Décrire un algorithme de détermination de la satisfiabilité d'une formule ϕ qui fonctionne par « retour sur trace ».
5. Transcrire en Python l'algorithme que vous avez conçu.